

If you're learning **Node.js + Express + EJS** for backend development, this is roughly **95% of what a real developer uses daily**. You do **not** need to memorize every EJS option from the docs. Focus on rendering dynamic pages, layouts, partials, forms, and data passing. EJS is a simple templating engine that lets you write JavaScript inside HTML templates. [\[ejs.co+1\]](#)

1. What is EJS?

EJS = **Embedded JavaScript**

Instead of serving static HTML:

HTML<h1>Welcome</h1>

You can inject dynamic data:

ejs<h1>Welcome <%= username %></h1>

Express sends data → EJS generates HTML → Browser receives normal HTML.
[\[ejs.co+1\]](#)

2. Installation

Bashnpm install express ejs

Set EJS as template engine:

```
JavaScriptconst express = require("express");
const app = express();
```

```
app.set("view engine", "ejs");
```

```
JavaScriptapp.get("/", (req, res) => {
  res.render("home");
});
```

Folder structure:

```
project/
├── views/
│   └── home.ejs
└── app.js
```

3. Passing Data

Server

```
JavaScriptapp.get("/", (req, res) => {
  res.render("home", {
    name: "Gurparshad",
    age: 21
  });
});
```

EJS

```
ejs<h1><%= name %></h1>
<p><%= age %></p>
```

Output:

```
HTML<h1>Gurparshad</h1>
<p>21</p>
```

4. EJS Tags (VERY IMPORTANT)

Output data

```
ejs<%= name %>
```

Escapes HTML for safety. [\[GeeksforGeeks\]](#)

Raw HTML

```
ejs<%- htmlContent %>
```

```
JavaScriptres.render("home", {  
  htmlContent: "<h1>Hello</h1>"  
});
```

Output:

HTML<h1>Hello</h1>

Not escaped. Use carefully. [\[GeeksforGeeks\]](#)

JavaScript Logic

```
ejs<% let x = 10; %>
```

No output.

5. Conditionals

```
ejs<% if(isLoggedIn) { %>
```

```
  <h1>Dashboard</h1>
```

```
<% } else { %>
```

```
  <h1>Please Login</h1>
```

```
<% } %>
```

This is used everywhere.

6. Loops

forEach

```
ejs<ul>
```

```
<% users.forEach(user => { %>
```

```
  <li><%= user %></li>
```

```
<% }) %>
</ul>
```

Output:

```
HTML<ul>
<li>John</li>
<li>Alex</li>
</ul>
```

for loop

```
ejs<% for(let i=0;i<5;i++) { %>

  <p><%= i %></p>

<% } %>
```

7. Arrays of Objects

Server:

```
JavaScriptconststudents= [
  {name:"John", marks:90},
  {name:"Alex", marks:85}
];
```

```
res.render("students",{students});
```

EJS:

```
ejs<% students.forEach(student => { %>

  <h3><%= student.name %></h3>
  <p><%= student.marks %></p>

<% }) %>
```

This is probably the most common pattern.

8. Includes (Partials)

Very important for large projects. [\[ejs.co\]](https://ejs.co)

header.ejs

```
ejs<nav>
  Navbar
</nav>
```

home.ejs

```
ejs<%- include("partials/header") %>
```

```
<h1>Home Page</h1>
```

Folder:

```
views/
|
├── home.ejs
├── partials/
│   └── header.ejs
```

9. Navbar Reuse

```
views/
|
├── partials/
│   ├── navbar.ejs
│   └── footer.ejs
└── home.ejs
```

```
ejs<%- include("partials/navbar") %>
```

```
<h1>Home</h1>
```

```
<%- include("partials/footer") %>
```

Real projects do this constantly.

10. Rendering Objects

Server:

```
JavaScriptconst user = {  
  name: "John",  
  age: 22  
};
```

```
res.render("profile", {user});
```

EJS:

```
ejs<h1><%= user.name %></h1>  
<p><%= user.age %></p>
```

11. Forms with EJS

```
ejs<form action="/login" method="POST">
```

```
  <input type="text" name="username">
```

```
  <button>Login</button>
```

```
</form>
```

Express:

```
JavaScriptapp.post("/login", (req, res) => {  
  console.log(req.body);  
});
```

Used heavily with authentication.

12. Dynamic Links

```
ejs<a href="/users/<%= user.id %>">  
  View User  
</a>
```

Output:

HTML<ahref="/users/1">

13. Dynamic Classes

```
ejs<div class="<%= isActive ? 'active' : " %>">
</div>
```

14. Ternary Operator

```
ejs<h1>
<%= isLoggedIn ? "Welcome" : "Login First" %>
</h1>
```

15. Layout Pattern

Without layout:

```
ejsNavbar
Page
Footer
```

repeated in every file ❌

Using includes:

```
ejs<%- include("partials/header") %>
```

Page Content

```
<%- include("partials/footer") %>
```

✅ Much cleaner.

16. Serving Static Files

```
JavaScriptapp.use(express.static("public"));
```

Folder:

```
public/  
|  
├── css  
├── js  
└── images
```

EJS:

```
ejs<link rel="stylesheet" href="/css/style.css">
```

17. Error Page

```
JavaScriptapp.use((req,res)=>{  
  res.status(404).render("404");  
});
```

404.ejs

```
ejs<h1>Page Not Found</h1>
```

18. What Happens Internally?

Request:

GET /

↓

Express Route

```
JavaScriptres.render("home",{name:"John"})
```

↓

EJS


```
ejs<h1><%= name %></h1>
```

↓

Generated HTML

```
HTML<h1>John</h1>
```

↓

Browser

This server-side rendering flow is the primary use case of EJS with Express.

[\[CodeForGeek+1\]](#)

19. Security (Important)

Safe:

```
ejs<%= userInput %>
```

Escapes HTML. [\[GeeksforGeeks\]](#)

Dangerous:

```
ejs<%- userInput %>
```

Can inject scripts if input is not trusted. [\[GeeksforGeeks\]](#)

Rule:

Use <%= 99% of the time

Use <%- only for trusted HTML

20. What a MERN Developer Should Actually Know

Master these:

☒ Passing data to views

✓ <%= %>

✓ <%- %>

✓ if-else

✓ loops

✓ arrays of objects

✓ includes/partials

✓ forms

✓ dynamic links

✓ static files

✓ layout pattern

✓ rendering database data

That's enough to build:

- Blog
- Student Management System
- Authentication App
- Todo App
- E-commerce SSR pages
- Admin Dashboard

Everything beyond this is advanced EJS and rarely needed in day-to-day Express development. 🚀